

第3章: Web基礎・HTML・CSS — 画面の骨組みと見た目（完全版）

この章では、Webページが表示されるしくみと、画面の **骨組みを作るHTML・見た目を整えるCSS** を、**本アプリの実際のスタイルファイルを1行残らず読み解きながら** 学びます。本アプリのフロント（前面）は React（リアクト）で書かれていますが、ブラウザが最終的に画面へ描き出すのは結局 **HTMLとCSS** です。ここを理解すれば、「画面のこの文字を変えたい」「この色を変えたい」「この余白を詰めたい」を、自分の手で安全にできるようになります。

プログラミング完全初心者の方を想定しています。専門用語は出てくるたびに「用語（よみ：意味）」の形でかみ砕き、たとえ話を多用します。コードは **すべて本アプリの実物** を引用し、説明のために作った簡易例には「※説明用の簡易例」と明記します。

この章は長いですが、**辞書のように使えること** を目指しています。実務で「あのプロパティ何だっけ」「この菱形ボタンどうやって作ってるんだっけ」と思ったら、ここに戻って該当箇所を引いてください。

この章で学ぶこと

- **ブラウザが画面を作るしくみ** — HTTPでHTMLを受け取り、DOM（ドム）を組み立て、CSSを当てて画面へ描く（レンダリング）までの流れ
- **HTMLの全体** — タグ・属性・入れ子・空要素・セマンティック（意味のある）タグ・よく使うタグ早見表
- **HTMLフォーム** — `form` / `label` / `input`（全type） / `button` / `select` / `textarea`。本アプリは入力フォームの塊
- **Login.tsx のJSXを1行ずつ完全解説** — `class` → `className`、`for` → `htmlFor`、`{ }`での値の埋め込み
- **CSSの全体** — セレクタ・カスケード（優先順位）・ボックスモデル・単位・色・Flexbox・擬似クラス/擬似要素・トランジション・アニメーション・メディアクエリ
- **本アプリの `global.css` を全342行、1行ずつ解説** — リセット・CSS変数・中央寄せレイアウト・色違いボタン7種・表・アラート・日付入力の調整
- **本アプリの `Login.module.css` を全100行、1行ずつ解説** — 出現アニメーション・入力フォーカス・菱形ヘルプボタン

- **KosuList / MainMenu / Pagination の実CSS** — 一覧画面・メニュー画面・共通部品の実例
- **CSS Modules (シーエスエス・モジュールズ)** — 名前衝突を防ぐしくみ
- **開発者ツール (F12)** — 見た目調整・調査の実践手順
- **アクセシビリティ・よくある落とし穴・トラブルシューティング・演習問題**

目次

0. 前提知識：ブラウザが画面を作るしくみ (DOM・レンダリング)
 1. HTMLの基本
 2. HTMLフォーム (本アプリの心臓)
 3. 実例：Login.tsx のJSXを1行ずつ完全解説
 4. CSSの基本文法
 5. ボックスモデル (余白の正体)
 6. 単位と色
 7. Flexbox (配置の主演)
 8. 擬似クラスと擬似要素
 9. トランジションとアニメーション
 10. メディアクエリ (レスポンシブ)
 11. 実コード全解説：global.css 全342行を1行ずつ
 12. 実コード全解説：Login.module.css 全100行を1行ずつ
 13. 実コード追加解説：KosuList / MainMenu / Pagination
 14. CSS Modulesのしくみ
 15. 開発者ツール (F12) 実践
 16. アクセシビリティ
 17. よくある落とし穴とトラブルシューティング
 18. 演習問題
 19. この章のまとめ
-

0. 前提知識：ブラウザが画面を作るしくみ（DOM・レンダリング）

「この章のコードを書き換えると、なぜ画面が変わるのか」を理解するために、まず **ブラウザ（Chrome・Edge など）の内部** を簡単にのぞいておきます。ここが分かると、後のCSS解説が「ただの呪文」ではなく「ちゃんと意味のある指示」に見えてきます。

0.1 そもそも「Webページが見える」とは何が起きているのか

ブラウザ（browser：ブラウザ）とは？ Webページを見るためのソフト。Google Chrome（グーグル・クローム）、Microsoft Edge（マイクロソフト・エッジ）、Safari（サファリ）などがあります。本アプリも、従業員はブラウザで開いて使います。

サーバー（server：サーバー）とは？ ページのデータ（HTMLや画像）を保管し、ブラウザから「ください」と言われたら渡すコンピュータ。本アプリでは、開発中は `http://localhost:3000`（フロント）や `http://localhost:8000`（バックエンド）が「自分のパソコンの中のサーバー」です。

あなたが `http://localhost:3000` を開くと、ブラウザは次の手順を踏みます。

1. サーバーに「このURLのページをください」とお願いする（HTTPリクエスト）
2. サーバーから HTML文書（ただの長い文字列）を受け取る（HTTPレスポンス）
3. HTMLを解析して「DOMツリー」という木構造のデータに組み立てる
4. CSS（見た目の指示）を解析して「どの要素にどんな見た目を当てるか」を決める
5. DOM + CSS を合体させて「レンダリングツリー」を作り、画面にピクセルとして描く（レンダリング）
6. JavaScript が DOM を書き換えると、3～5 がやり直され、画面が更新される

HTTP（エイチティーティーピー：HyperText Transfer Protocol）とは？ ブラウザとサーバーが会話するときの「言葉のルール（プロトコル）」。「このページください（GET）」「このデータ保存して（POST）」といったお願いの形式が決まっています。本アプリの通信もこのHTTPで行われます（詳しくは第4章・第7章）。

レンダリング（rendering：レンダリング）とは？「文字や色や配置の指示」を、実際に目に見える絵（ピクセル）として画面に描き出すこと。日本語では「描画」とも言います。CSSを変えると見た目が変わるのは、この再レンダリングが起きるからです。

0.2 DOM（ドム） — HTMLが「木」になる

DOM（ドム：Document Object Model）とは？ HTMLを、プログラムから操作できる「木構造（ツリー）のオブジェクト」として表したものの。「Document（文書）」を「Object（もの）」の「Model（模型）」にしたもの、という意味です。

たとえば次のHTMLは…

```
<div>
  <h1>タイトル</h1>
  <p>本文</p>
</div>
```

ブラウザの中では、次のような「木」になります。

```
div （幹）
├─ h1 （枝）  "タイトル"
└─ p  （枝）  "本文"
```

家系図や組織図と同じで、**親（parent）・子（child）・兄弟（sibling）** の関係があります。上の例では `div` が親、`h1` と `p` が子、`h1` と `p` どうしは兄弟です。

なぜ「木」にするの？ 木の形にしておくと、プログラム（JavaScript）が「この `h1` の文字を変えて」「この `div` の中に新しい `p` を足して」と、ピンポイントで操作できるからです。Reactは、まさにこのDOMを **JavaScriptで効率よく書き換える** ことで、ページ全体を読み込み直さずに画面の一部だけを更新します（第5章で詳述）。

0.3 3つの言語の役割（再確認）

種類	役割	家でたとえると	本アプリの該当ファイル
HTML	構造・内容（何があるか）	骨組み（柱・壁・部屋割り）	<code>.tsx</code> 中の JSX
CSS	見た目（どう見えるか）	内装・塗装・家具の配置	<code>styles/</code> 中の <code>.css</code>
JavaScript / TypeScript	動き（何が起きるか）	電気・スイッチ・設備	<code>.tsx</code> の処理部分

この章はHTMLとCSSに集中します。JavaScript / TypeScript は第4章・第5章で扱います。

用語: TypeScript（タイプスクリプト）とは？ JavaScript（ジャバスクリプト）に「型（かた）」というルールを足した言語。本アプリのフロントは `.tsx` という拡張子のファイルで、これは「TypeScript + JSX」という意味です。詳しくは第4章。

1. HTMLの基本

HTML（エイチティーエムエル：HyperText Markup Language）とは？ 「タグ」という印で文書に意味のラベルを貼り、構造を表す言語です。「Markup（マークアップ）」は「印を付ける」という意味。文章のどこが見出しで、どこが段落で、どこがボタンなのかを、タグで示します。

1.1 タグの形 — 開始・中身・終了

```
<p>これは段落です。</p>
```

分解すると：

- `<p>` … **開始タグ**（「ここから段落（paragraph）」の合図）

- **これは段落です。** … **中身（コンテンツ）**
- `</p>` … **終了タグ**（スラッシュ `/` が付く。「段落ここまで」の合図）

たとえ: タグは「付箋」のようなものです。 `<p>` という付箋を貼り始め、 `</p>` で貼り終わる。挟まれた範囲が「段落」という意味になります。

タグは **入れ子（ネスト）** にできます。外側のタグの中に内側のタグを入れます。

```
<div>                                <!-- 外側の箱 -->
  <h1>見出し</h1>                  <!-- 内側：見出し -->
  <p>本文</p>                       <!-- 内側：段落 -->
</div>                              <!-- 外側の箱おわり -->
```

用語: コメント `<!-- ... -->` HTMLの中にも書いても画面に出ない「メモ書き」。人間向けの注釈です。上の `<!-- 外側の箱 -->` がそれです。

⚠ **入れ子のルール:** 開始と終了は **必ず対応** させ、交差させてはいけません。 `<a><b></a></b>` は誤り（aとbが交差している）、 `<a><b></b></a>` が正しい（bがaの中に完全に収まっている）。交差させると、ブラウザが意図しない形に直してしまい、表示が崩れます。

1.2 空要素（終了タグがないタグ）

中身を持たないタグは、終了タグがありません。これを **空要素（くうようそ）** と呼びます。

```
    <!-- 画像。中身なし -->
<input type="text">                 <!-- 入力欄。中身なし -->
<br>                                <!-- 改行 -->
<hr>                                <!-- 水平線 -->
```

なぜ終了タグがないの？ これらは「中に文章を挟む」必要がないからです。画像や入力欄は、それ自体が1つの部品で、間に何かを入れる余地がありません。

⚠ **JSX (React) では空要素も必ず閉じる:** Reactの世界では、空要素も末尾に `/` を付けて `<img />` `<input />` のように「自己完結」させる必要があります。これを忘れるとエラーになります (§3で実例を見ます)。

1.3 属性 (attribute)

タグに追加情報を与えるのが **属性 (attribute: アトリビュート)**。 **属性名="値"** の形で **開始タグの中** に書きます。

```
<a href="/help" target="_blank" class="link">ヘルプ</a>
<!-- href=リンク先   target=開き方   class=スタイル用の名札   -->
```

代表的な属性：

属性	意味	例
<code>id</code>	要素に1つだけ付ける固有の名札	<code>id="numberInput"</code>
<code>class</code>	スタイル用の名札 (複数要素で共有可)	<code>class="blue_button"</code>
<code>href</code>	リンク先URL (<code><a></code> 用)	<code>href="/login"</code>
<code>src</code>	読み込むファイルの場所 (画像など)	<code>src="logo.png"</code>
<code>alt</code>	画像が出ないときの代替文字	<code>alt="ロゴ"</code>
<code>type</code>	入力欄やボタンの種類	<code>type="number"</code>
<code>value</code>	入力欄の値	<code>value="123"</code>
<code>required</code>	入力必須 (値なし属性)	<code>required</code>
<code>disabled</code>	操作不可 (値なし属性)	<code>disabled</code>
<code>role</code>	要素の「役割」を支援技術に伝える	<code>role="alert"</code>

用語: id と class の違い `id` (アイディー) は「1ページに1つだけ」の固有名 (マイナンバーのようなもの)。`class` (クラス) は「同じ見た目を複数の要素に付ける」ための共有名 (制服のようなもの)。CSSでは `id` は先頭に `#`、`class` は先頭に `.` を付けて指定します (§4)。

用語: 値なし属性 (boolean属性) `required` や `disabled` は値を書かず、書くだけで「オン」になります。 `required` と書けば「必須」、書かなければ「任意」。スイッチのようなものです。

1.4 よく使うタグ早見表

```
<h1>~<h6>    <!-- 見出し (h1が最大、h6が最小) -->
<p>            <!-- 段落 (paragraph) -->
<div>          <!-- 意味を持たない箱 (レイアウト用)。最頻出 -->
<span>         <!-- 行内の小さな箱 (文章の一部だけを囲む) -->
<a href="">    <!-- リンク (anchor) -->
<img src="">   <!-- 画像 (image) -->
<ul><li>       <!-- 順序なしリスト (箇条書き) -->
<ol><li>       <!-- 順序ありリスト (番号付き) -->
<table>        <!-- 表 -->
  <thead><tbody> <!-- 表のヘッダ部・本体部 -->
  <tr>          <!-- 行 (table row) -->
  <th>          <!-- 見出しセル (table header) -->
  <td>          <!-- データセル (table data) -->
<nav>          <!-- ナビゲーション (メニュー) の領域 -->
<form>         <!-- 入力フォームの枠 -->
<label>        <!-- 入力欄の見出しラベル -->
<input>        <!-- 入力欄 -->
<button>       <!-- ボタン -->
<select><option> <!-- ドロップダウン選択 -->
<textarea>     <!-- 複数行の入力欄 -->
```

用語: セマンティックHTMLとは? 「セマンティック (semantic)」は「意味のある」という意味。 `<div>` ばかり使うのではなく、 `<nav>` (メニュー)・`<table>` (表)・`<button>` (ボタン) のように **役割の分かるタグ** を使うこと。検索エンジンや、目の不自由な方が使う **読み上げソフト** が、内容を正しく理解できるようになります。本アプリも `<nav>` `<table>` `<form>` `<button>` を適切に使っています (§ 11・§ 13で確認)。

div と span の違い: どちらも「意味を持たない箱」ですが、 `<div>` は **ブロック要素** (縦に積み上がり、横幅いっぱい広がる)、 `<span>` は **インライン要素** (文章の途中に置け、必要な

分だけ幅をとる)。たとえば: `div` は「段落まるごとを囲む大きな袋」、`span` は「文章中の一単語だけにマーカーを引く」イメージ。

2. HTMLフォーム（本アプリの心臓）

本アプリは「工数入力」「人員登録」「ログイン」「問い合わせ」など **入力フォームの塊** です。ここを制すれば本アプリのHTMLの大半が読めます。

用語: フォーム（form：フォーム）とは？ ユーザーが文字や数値を入力し、サーバーへ送るための「申込用紙」のようなまとまり。入力欄・ラベル・送信ボタンをひとまとめにします。

2.1 フォームの基本構造

▼ **このコードがやること（先に日本語で）**：「従業員番号」というラベルの付いた数値入力欄と、「ログイン」という送信ボタンを、1つのフォーム（申込用紙）にまとめています。

```
<form>                                <!-- 入力をまとめる枠。送信の単位 -->
  <label for="numberInput">従業員番号</label>  <!-- 見出しラベル -->
  <input type="number" id="numberInput">        <!-- 入力欄 -->
  <button type="submit">ログイン</button>      <!-- 送信ボタン -->
</form>
```

- `<form>` … 入力欄をまとめる枠。中の送信ボタンを押すと「送信（submit）」というイベント（出来事）が起きます。
- `<label for="ID">` … 入力欄の見出し。 `for` を `<input>` の `id` と **一致** させると、ラベルの文字をクリックしただけで、その入力欄にカーソルが入ります（クリックしやすくなる）。
- `<button type="submit">` … フォームを送信するボタン。

2.2 input の type（種類）一覧

`type` を変えると、入力欄の見た目と挙動がガラッと変わります。本アプリで使うものを中心に：

type	用途	見た目・挙動	本アプリでの例
text	一般の文字	ふつうの一行入力	氏名など
number	数値	右端に上下ボタンが付く	従業員番号
date	日付	カレンダーが開く	就業日・検索日
checkbox	オン/オフ	四角いチェックボックス	権限・各種フラグ
password	伏せ字	入力が●●●になる	(必要に応じて)
radio	複数から1つ選ぶ	丸いラジオボタン	選択肢

⚠ `<input type="number">` の注意: カレンダー型や数値型でも、入力された値はJavaScript側では **文字列** として届きます。だから `Login.tsx` では `Number(employee_no)` と数値に変換しています (§ 3⑩、詳しくは第4章)。「画面の数字」と「プログラムが受け取る数字」は別物だと意識してください。

2.3 select（ドロップダウン）と textarea（複数行入力）

▼ このコードがやること: 上は「PかRを選ぶドロップダウン」、下は「複数行の文章を入れる欄」です。

```
<select>                                <!-- ドロップダウン選択 -->
  <option value="P">P</option>          <!-- 選択肢1 -->
  <option value="R">R</option>          <!-- 選択肢2 -->
</select>

<textarea rows="4"></textarea>         <!-- 複数行入力（問い合わせ本文など）。rows=表示する行数 -->
```

本アプリのショップ選択（`ShopSelect`）や問い合わせ入力（`InquirNew`）で使われています。

`<option value="...">` の `value` が「実際に送られる値」、タグの中身が「画面に見える文字」です。

3. 実例：Login.tsx のJSXを1行ずつ完全解説

ここで、本アプリのログイン画面 `frontend/src/MainPage/Login.tsx` を、HTMLの知識で読みます。このファイルは「画面を作る部分（JSX）」と「動きを作る部分（JavaScript）」が混ざっています。本章では **画面部分** を中心に、1行ずつ追います（動きの部分は第4章・第5章で深掘り）。

用語: JSX（ジェイエスエックス）とは？ JavaScript（TS）の中に、HTMLのような記述を混ぜて書く書き方。Reactで画面を書く記法です。HTMLとの主な違いは次の3点：

- `class` → `className`（「class」はJavaScriptで別の意味の予約語なので避ける）
- `for` → `htmlFor`（同様に「for」も予約語）
- `{ }` の中に **JavaScriptの値や処理** を埋め込める

3.1 ファイル冒頭の import（読み込み）部分

▼ **このコードがやること:** この画面で使う部品（React本体、ページ移動の道具、通信の道具、ロゴ画像、専用CSS）を外部から読み込んでいます。「材料を仕入れる」段階です。

```
// ① React本体と、状態を持つ道具 useState
import React, { useState } from "react";
// ② 画面間リンクと、画面移動の道具
import { Link, useNavigate } from "react-router-dom";
// ③ サーバーと通信する道具（第4章）
import api from "../api/axios";
// ④ 通信ライブラリ本体
import axios from "axios";
// ⑤ ロゴ画像を読み込んで logo という名前に
import logo from "../img/TitleRogo.png";
// ⑥ この画面専用のCSS（§12で全解説）
import styles from "../styles/MainPage/Login.module.css";
```

- ⑤ … 画像も `import` で読み込みます。後で `<img src={logo} />` のように使います。
- ⑥ … `Login.module.css` を `styles` という名前で読み込み。 `styles["login-wrapper"]` のように、CSSのクラスを取り出して使います（**CSS Modules**。§14で詳述）。

3.2 画面を作って返す部分（JSX本体）

▼ このコードがやること（先に日本語で）：画面全体を `login-wrapper` という箱で囲み、その中に「ロゴ画像」「ログインフォーム（見出し・従業員番号の入力欄・エラー表示・ログインボタン）」「ヘルプボタン」を縦に並べています。これがログイン画面の見た目の正体です。

```

return (
  /* ① 画面全体を囲む箱。Modulesのクラスを適用 */
  <div className={styles["login-wrapper"]}>
    /* ② ロゴ画像。src=画像、alt=代替文字 */
    <img src={logo} alt="業務工数システムロゴ"
      className={styles["login-logo"]} />
    /* ③ フォーム枠（申込用紙） */
    <form
      /* 送信時に handleSubmit を呼ぶ（第4章） */
      onSubmit={handleSubmit}
      /* キー入力時の処理（Enterの暴発防止） */
      onKeyDown={(e) => {
        if (e.key === "Enter" && e.target instanceof HTMLInputElement
          && e.target.type !== "textarea") {
          /* Enterでの誤送信を止める */
          e.preventDefault();
          /* 入力欄からフォーカス（カーソル）を外す */
          (e.target as HTMLInputElement).blur();
        }
      }}
    >
      /* ④ 見出し（global.cssで中央寄せ） */
      <h2>ログイン</h2>
      /* ⑤ 入力欄をまとめる箱（縦並び） */
      <div className={styles["search-bar"]}>
        /* ⑥ ラベル。htmlForを下のidと一致 */
        <label htmlFor="numberInput">従業員番号</label>
        <input
          /* ⑦ 数値入力欄 */
          type="number"
          /* ラベルと結ぶ id */
          id="numberInput"
          /* 表示する値（stateと連動。第5章） */
          value={employee_no}
          /* 入力が変わるたびに */
          onChange={(e) => {
            const value = e.target.value;
            /* stateを更新（空なら空、なければ値） */
            setNumber(value === "" ? "" : value);
          }}
          /* ⑧ 入力必須 */
          required
          className={styles["input-focus"]}
        />
        /* ⑨ エラーがあるときだけ表示（&&） */
        {errorMessage && (
          /* role="alert"は支援技術への通知（§16） */
          <div role="alert">{errorMessage}</div>
        )}
      </div>
    </div>
  )

```

```

    })

    { /* ⑩ 送信ボタン。global.cssの青ボタン */
    <button type="submit" className="blue_button">ログイン</button>
    </div>
  </form>
  { /* ⑪ ヘルプへのリンク（菱形ボタン。§12） */
  <Link to="/help" className={styles["help-button"]} ></Link>
  </div>
);

```

各番号の解説：

- ①⑤ … `styles["..."]` は **CSS Modules** (§14)。`Login.module.css` のクラスを当てています。`styles["login-wrapper"]` は「Login.module.css の `.login-wrapper` ルールを適用せよ」という意味です。
- ② … `<img />` は空要素なので末尾を `/>` で閉じます。`src={logo}` の `{ }` は「JavaScriptの値（先ほど import した logo 画像）を入れる」記法。`alt="業務工数システムロゴ"` は、画像が表示できないときや読み上げ時の代替テキスト (§16のアクセシビリティ)。
- ③ … `<form>` に `onSubmit` (送信時の処理) と `onKeyDown` (キー入力時の処理) を結びつけています。`onoo` で始まる属性は「〇〇が起きたら、この処理を実行せよ」という **イベントハンドラ** (第5章で詳述)。
- ④ … `<h2>` は見出し。`global.css` の `h2` ルールで中央寄せ&サイズ指定されます (§11.6)。
- ⑥ … `htmlFor="numberInput"` を、下の `<input>` の `id="numberInput"` と一致させています。これで「従業員番号」の文字をクリックすると入力欄にカーソルが入ります。HTMLの `for` がJSXでは `htmlFor` になる実例です。
- ⑦⑧ … `type="number"` で数値入力欄、`required` で入力必須に。
- ⑨ … `{errorMessage && (<div role="alert">...)}` は「`errorMessage` に中身があるときだけ、後ろの要素を表示」という条件表示 (第5章 §2)。エラー時だけ赤いアラート (§11.11で全解説) が出ます。`role="alert"` は読み上げソフトへの「これは警告です」という合図。
- ⑩ … `className="blue_button"` (**文字列で直接**) は `global.css` (§11) の共通青ボタン。同じ画面で「Modules (`styles[...]`)」と「global (文字列)」が混在しているのがポイントです。
- ⑪ … `<Link to="/help">` はReact Router (第5章) の画面内リンク。見た目は `help-button` クラス (45度傾いた菱形。§12で全解説)。中身が空なのは、文字ではなくCSSの `::after` で「?」を描いているからです。

このように、1つのログイン画面でも「**JSX (HTML構造) + CSS Modules + global.css + Reactの処理**」が組み合わさっています。次は、この見た目を作っているCSSを徹底的に読み

ます。

4. CSSの基本文法

用語: CSS（シーエスエス：Cascading Style Sheets）とは？ HTMLの **見た目** を指定する言語。「Cascading（カスケーディング）」は「滝のように上から流れて重なる」という意味で、複数の指示が重なったときの優先順位のしくみを表します（後述）。

4.1 ルールの形

```
セクタ {  
  プロパティ: 値;  
  プロパティ: 値;  
}
```

具体例（※説明用の簡易例）：

```
/* セクタ：対象（すべての p） */  
p {  
  /* プロパティ:値 → 文字色を赤に。最後は必ず ; */  
  color: red;  
  /* 文字サイズ16ピクセル */  
  font-size: 16px;  
}
```

- **セクタ（selector）** … 「誰に」スタイルを当てるかの指定（上の例では「すべての `<p>`」）。
- **プロパティ（property）** … 「何を」変えるか（色、サイズ、余白…）。
- **値（value）** … 「どう」するか（赤、16px…）。
- 各行の末尾には **必ずセミコロン ;** を付けます。これを忘れると、その行から次の行までが壊れます（よくあるミス）。

4.2 セレクタ（対象の指定）一覧

書き方	意味	例
<code>p</code>	タグ名で指定	すべての <code><p></code>
<code>.box</code>	クラスで指定（先頭ドット）	<code>class="box"</code> の要素
<code>#header</code>	IDで指定（先頭シャープ）	<code>id="header"</code> の要素
<code>.box p</code>	子孫（box の中のすべての p）	
<code>.box > p</code>	直接の子のみ	
<code>a:hover</code>	擬似クラス（マウスを乗せた時）	§ 8
<code>input[type="date"]</code>	属性で指定（属性セレクタ）	§ 11で実例
<code>div[role="alert"]</code>	属性の値まで指定	§ 11.11で実例
<code>*</code>	すべての要素（全称セレクタ）	§ 11.1で実例
<code>a, p</code>	複数まとめて（カンマ区切り）	
<code>th, td</code>	表のヘッダとデータ両方	§ 11.9で実例

4.3 カスケード（優先順位） — 同じ要素に複数のルールが当たったら

用語: カスケード（cascade：カスケード）とは？ CSSの「C」。複数のルールが同じ要素に同時に当たったとき、**優先順位** に従って勝者が決まるしくみ。

優先順位はおおまかに次の順です（強い順）：

1. `!important` を付けたもの（最優先・乱用禁物）
2. インラインスタイル（`style="..."` で直接書いたもの）
3. `#id` で指定
4. `.class` や `[属性]` で指定
5. タグ名で指定

そして **同じ強さなら「後にした方」が勝ちます。**

⚠ **!important の乱用に注意:** `!important` を付けると最優先になりますが、これを多用すると「なぜか上書きできないルール」だらけになり、保守が地獄になります。本アプリでも `MainMenu.module.css` の `display: none !important;` (§ 13) など、ごく限られた場所でのみ使われています。

4.4 CSSの書き場所（3通り）

1. **外部ファイル**（推奨・本アプリの方式）：`.css` ファイルに書き、`import` で読み込む。
2. **インラインスタイル**：`<div style="color:red">`（その要素だけ。Reactでは `style={{color:'red'}}>`）。
3. **`<style>` タグ**：HTML内に直接。本アプリでは基本使いません。

本アプリは **外部ファイル**（`global.css` と各 `.module.css`）です。理由は「見た目をまとめて管理でき、複数画面で使い回せる」から。

5. ボックスモデル（余白の正体）

すべてのHTML要素は、**四角い箱**として扱われます。この箱の構造を「ボックスモデル」と呼び、CSSで最も重要な概念の1つです。「なぜか余白が消えない」「なぜか幅がはみ出す」の9割は、ここの理解不足が原因です。



- **content（コンテンツ）** … 中身（文字や画像）。`width`・`height` で大きさを指定。
- **padding（パディング）** … 枠の内側の余白。中身と枠の間。「箱の内張り」。
- **border（ボーダー）** … 枠線。
- **margin（マージン）** … 枠の外側の余白。他の箱との距離。「箱と箱のソーシャルディスタンス」。

覚え方: 内側から「中身→内張り(padding)→壁(border)→外の距離(margin)」。padding は「中身を枠から離す」、margin は「箱を他の箱から離す」。

5.1 box-sizing: border-box が超重要

▼ **なぜ重要か（先に日本語で）:** 既定では width（幅）は「中身だけの幅」を指し、padding と border は **その外側に追加** されます。つまり width:200px; padding:10px だと、見た目の実寸は $200 + 10 + 10 = 220\text{px}$ になり、計算が狂います。

これを防ぐのが **box-sizing: border-box**。これを指定すると width が「padding と border を **含めた** 全体幅」になり、width:200px と書けば見た目もちゃんと200pxになります。直感どおりになるわけです。

```
/* 全要素を「幅にpadding/borderを含める」方式に（※本アプリの実装。§11.1で再掲） */
* { box-sizing: border-box; }
```

本アプリは **global.css** の冒頭で **全要素に** これを適用しています（§11.1で実コード確認）。だから本アプリでは「幅の計算が狂う」事故が起きにくくなっています。

5.2 margin / padding のショートハンド（まとめ書き）

```
/* 上下左右すべて10px */
margin: 10px;
/* 上下10px / 左右20px */
margin: 10px 20px;
/* 上→右→下→左（時計回り） */
margin: 10px 20px 30px 40px;
/* 上下0、左右auto＝左右中央寄せ（頻出!） */
margin: 0 auto;
/* 上だけ個別指定 */
margin-top: 8px;
```

margin: 0 auto は「**横方向の中央寄せ**」の定番。左右を auto（自動）にすると、ブラウザが左右の余白を等しく取り、結果として要素が水平中央に来ます。本アプリの **login-logo** や

`form`、各種 `wrapper` で多用されています (§ 11・§ 12)。

6. 単位と色

6.1 大きさの単位

単位	意味	使いどころ
<code>px</code>	ピクセル（絶対的な点）	枠線・細かい余白
<code>%</code>	親要素に対する割合	幅を親の何%にするか
<code>rem</code>	ルート（html）の文字サイズ基準	文字サイズ（拡大縮小に強い）
<code>em</code>	その要素の文字サイズ基準	アイコンを文字に合わせる等
<code>vw</code> / <code>vh</code>	画面幅 / 高さの1%（viewport）	画面サイズに応じた大きさ

用語: viewport（ビューポート）とは？ 「今ブラウザで見えている表示領域」のこと。 `vw` は viewport width（表示幅）の1%、 `vh` は viewport height（表示高さ）の1%。 `100vh` なら「画面の高さいっぱい」。本アプリは `min-height: 100vh`（最低でも画面の高さ分は確保）で使っています (§ 11.6)。

6.2 便利な関数 — `clamp` / `calc` / `min`

本アプリは画面サイズへの対応のため、CSSの関数を多用します。

`clamp(最小, 推奨, 最大)` … 「推奨値を使うが、最小～最大の範囲に収める」関数。画面サイズに応じて文字を伸縮させつつ、小さすぎ・大きすぎを防ぎます。本アプリは見出しサイズに使っています (§ 11.5)：

```
--heading1: clamp(1.5rem, 1.5rem + 3vw, 4rem);  
/* 最小1.5rem、画面幅に応じて1.5rem+3vw、最大4rem */
```

calc(式) … CSSの中で計算ができる関数。 **calc(100% - 60px)** なら「親の幅いっぱいから60px引いた値」。本アプリのログイン入力欄の幅に使っています (§ 12.4)。

min(値A, 値B) … 「2つのうち小さい方」を採る関数。 **min(400px, 90%)** なら「大画面では400px、小画面では画面の90%」。ロゴのはみ出し防止に使っています (§ 12.3)。

6.3 色の指定

```
/* 色名（限られた数） */
color: red;
/* 16進数 #RRGGBB（R赤 G緑 B青 を00～ffで） */
color: #007bff;
/* 3桁省略形（#00ff00 と同じ＝緑） */
color: #0f0;
/* 赤緑青を0～255で */
color: rgb(0, 123, 255);
/* 最後は透明度（0=透明～1=不透明） */
color: rgba(0, 0, 0, 0.1);
```

16進数カラーの読み方: **#007bff** は「赤=00、緑=7b、青=ff」。 **#0f0** は短縮形で **#00ff00**（赤0・緑最大・青0＝純緑）と同じ。本アプリは主に **16進数**（**#007bff** 青、**#0f0** 緑、**#0ff** 水色 など）を使い、影の透明度には **rgba** を使っています (§ 11)。

7. Flexbox（配置の主役）

要素を **横並び・縦並び・中央寄せ** するための、現代CSSの主役です。本アプリのレイアウトはほぼすべてFlexboxで作られています。

用語: Flexbox（フレックスボックス）とは？ 「Flexible Box Layout（柔軟な箱配置）」の略。親要素に **display: flex** を付けると、その中の子要素を自由に並べられるしくみ。「子要素を整列させる魔法のトレイ」とイメージすると分かりやすいです。

7.1 基本

親要素に `display: flex` を付けると、その **直接の子要素** が「フレックスアイテム」として並びます。

```
.container {  
  /* フレックス開始（この箱の子を整列させる） */  
  display: flex;  
  /* 並ぶ向き：row=横（既定），column=縦 */  
  flex-direction: row;  
  /* 主軸（並ぶ向き）の揃え */  
  justify-content: center;  
  /* 交差軸（直角方向）の揃え */  
  align-items: center;  
  /* アイテム間の隙間 */  
  gap: 15px;  
  /* 入りきらない時に折り返す */  
  flex-wrap: wrap;  
}
```

7.2 justify-content と align-items の値

- **justify-content（主軸方向の揃え）**：`flex-start`（先頭寄せ） / `center`（中央） / `flex-end`（末尾寄せ） / `space-between`（両端 + 等間隔） / `space-around`（各アイテムの周りに等間隔）。
- **align-items（交差軸方向の揃え）**：`stretch`（既定・引き伸ばし） / `center`（中央） / `flex-start` / `flex-end`。

覚え方（重要）：`flex-direction: row`（横並び）のとき、**justify-content** は **横方向** の揃え、**align-items** は **縦方向** の揃え。`column`（縦並び）にすると、この2つの担当方向が **入れ替わります**。「中央寄せできない！」の原因は、たいていこの軸の取り違いです。

本アプリの `search-bar`（ログインの入力エリア）は、`column`（縦並び） + 中央寄せ + 隙間15pxで作られています（§12.2で実コード確認）。一方、一覧画面の `KosuList` の `search-bar` は `row`（横並び）です（§13）。同じ名前でも画面ごとに方向が違うのが面白いところです。

8. 擬似クラスと擬似要素

8.1 擬似クラス（状態に応じたスタイル）

要素の「状態」に応じてスタイルを変えます。 `:`（コロン）を **1つ** 付けます。

```
/* マウスを乗せている時 */
a:hover { }
/* 入力欄が選択（カーソルが入っている）時 */
input:focus { }
/* 偶数番目の行 */
tr:nth-child(even) { }
/* 操作不可の時 */
button:disabled { }
```

用語: hover（ホバー） / focus（フォーカス）

- `:hover` … マウスポインタを乗せている状態。ボタンの色変化などに使う。
- `:focus` … 入力欄などが「選択されてキー入力を受け付ける」状態。枠を光らせる演出に使う。

本アプリは `:hover`（ボタンの色変化）と `:focus`（入力欄の枠を青く光らせる）を多用します（§11・§12・§13）。`:disabled` はページ送りの「これ以上戻れない」ボタンに使われています（§13のPagination）。

8.2 擬似要素（要素の一部を作る）

要素の前後に「飾りの中身」を追加します。 `::`（コロン）を **2つ** 付けます。

```
/* 要素の中身の「前」に文字を足す */
.icon::before { content: "⚠ "; }
/* 要素の中身の「後」に文字を足す */
.icon::after { content: "?"; }
```

- `content` プロパティが **必須**（足す中身を指定。無いと表示されない）。
- 本アプリの **ヘルプボタンの「？」マーク**（§12.5）や、**アラートの「⚠」**（§11.11）は、この `::after` / `::before` で作られています。HTMLには書かず、CSSだけで文字を「生やして」い

るわけです。

9. トランジションとアニメーション

9.1 トランジション（なめらかな変化）

状態が変わるとき、見た目を **じわっと** 変化させます。

```
.blue_button {
  background-color: #007bff;
  /* 背景色を0.3秒かけて変化 */
  transition: background-color 0.3s ease;
}
.blue_button:hover {
  /* ホバーで濃い青へ（0.3秒かけてじわっと） */
  background-color: #0056b3;
}
```

- **transition**: **対象プロパティ** **時間** **変化の緩急**; の形。
- **ease**（イーズ）は「最初ゆっくり→速く→最後ゆっくり」という自然な緩急。他に **linear**（一定速度）などがあります。
- これが無いと、ホバーの色変化が「パツ」と一瞬で切り替わります。**transition** があると「ふわっ」と気持ちよく変わります。本アプリの全ボタンに付いています（§ 11.10）。

9.2 キーフレームアニメーション（自動で動く）

@keyframes（キーフレーム）で「開始→終了」の変化を定義し、**animation** で再生します。

```

/* fadeInUp という名前のアニメを定義 */
@keyframes fadeInUp {
  /* 開始：透明 & 50px下 */
  from { opacity: 0; transform: translateY(50px); }
  /* 終了：不透明 & 定位置 */
  to   { opacity: 1; transform: translateY(0); }
}
.login-wrapper {
  /* 上のアニメを1秒で再生（下からふわっと出現） */
  animation: fadeInUp 1s ease-out;
}

```

- `opacity`（オパシティ）… 不透明度（0=透明～1=不透明）。
- `transform: translateY(50px)` … 縦方向に50px移動（Y軸＝縦）。`translateX` なら横。
- `from` / `to` … アニメの「最初の状態」と「最後の状態」。
- 本アプリのログイン画面・一覧画面・メニュー画面は、これで **下からふわっと現れる** 演出をしています（§12・§13で実コード解説）。同じ `fadeInUp` が複数ファイルにコピーされているのも本アプリの特徴です。

10. メディアクエリ（レスポンス）

画面の幅に応じてスタイルを切り替え、PC・タブレット・スマホのいずれでも見やすくします。

```

/* 画面幅が768px以下のときだけ適用 */
@media (max-width: 768px) {
  form {
    width: auto;
    /* 最小幅の指定を解除（小さい画面で横はみ出し防止） */
    min-width: unset;
  }
}

```

- `@media (条件) { ... }` 中のルールは、条件に合うときだけ効きます。
- `max-width: 768px` は「画面幅が768px **以下**」。スマホやタブレット縦向きが該当します。
- `unset`（アンセット）は「その指定をなかったことにする」キーワード。
- 本アプリは `global.css` ・ `KosuList.module.css` ・ `MainMenu.module.css` ・ `Pagination.module.css` でこの768px分岐を使っています（各§で確認）。

用語: レスポンシブデザイン (responsive design) とは? 1つのコードで、PCでもスマホでも見やすく表示を変える設計のこと。「responsive=反応する」。画面サイズに「反応して」レイアウトが変わります。メディアクエリと、`clamp()` などの伸縮単位で実現します。

11. 実コード全解説：global.css 全342行を1行ずつ

ここからが本章の核心です。本アプリの **全画面共通のスタイル** `frontend/src/styles/global.css` を、**最初から最後まで** 読みます。これが読めれば、本アプリの見た目の「土台」を完全に理解できます。

11.1 全要素リセット (1～5行目)

▼ **このブロックがやること:** すべての要素の初期の余白を消し、ボックスモデルを扱いやすい方式に統一します。「ブラウザごとの初期スタイルの違い」をならす、いわゆる **CSSリセット** です。

```
/* * は「すべての要素」を意味する全称セレクトア */
* {
  /* 全要素の外側余白を0に（ブラウザ既定の余白を消す） */
  margin: 0;
  /* 全要素の内側余白を0に */
  padding: 0;
  /* widthにpadding/borderを含める方式に統一（§5.1） */
  box-sizing: border-box;
}
```

なぜリセットするの? ブラウザは `<h1>` や `<p>` や `<body>` に、何も指定しなくても勝手に余白を付けます。しかもブラウザごとに微妙に違うため、まず全部0にしてから **自分で一から余白を設計** します。これがプロの定番手法です。 `box-sizing: border-box` を全要素に効かせているので、本アプリは「paddingを足したら幅がはみ出した」事故が起きにくくなっています。

11.2 html / body の基本（7～10行目）

```
html, body {  
  /* 画面の高さいっぱいを使えるように */  
  height: 100%;  
  /* 既定フォント。Arialが無ければゴシック系 */  
  font-family: 'Arial', sans-serif;  
}
```

- `html, body` … カンマで2つの要素をまとめて指定。
- `height: 100%` … `html` と `body` の高さを「親（＝表示領域）の100%」に。これで後の `min-height: 100vh` などが効きやすくなります。
- `font-family` … 文字の書体。複数指定し、左から「使えれば使う」。`sans-serif`（サンセリフ）は「ゴシック体系のどれか」という最後の保険（特定のフォントが無くて、必ず何かのゴシック体になる）。

11.3 フォーム部品のリセット（12～23行目）

▼ **このブロックがやること:** ボタン・入力欄・選択欄・複数行入力を持つ、ブラウザ既定の見た目（枠・背景・フォント）を消し、フォントを親から継承させて統一します。

```
button,  
input,  
select,  
textarea {  
  /* 親要素のフォントを引き継ぐ（部品だけ別フォントになるのを防ぐ） */  
  font-family: inherit;  
  /* 文字サイズも親基準に */  
  font-size: 100%;  
  margin: 0;  
  padding: 0;  
  /* 既定の枠線を消す（後で自分で付け直す） */  
  border: none;  
  /* 既定の背景を消す */  
  background: none;  
  /* クリック時の青い枠を消す（focusで自分で付ける） */  
  outline: none;  
}
```

用語: inherit (インヘリット) 「親要素の値を引き継ぐ」キーワード。 `<button>` や `<input>` は、何もしないと別フォント（ブラウザ既定）になりがちなので、ここで「親に合わせろ」と命じています。

⚠ `outline: none` で、入力欄をクリックしたときの「青い枠」を消しています。ただし消しっぱなしだと「今どこを選択中か」が分からなくなるので、本アプリは `:focus` で自前の枠を付け直しています (§ 12.4)。これは見た目とアクセシビリティのバランスをとった設計です。

11.4 リスト・リンク・画像のリセット (25~41行目)

```
ul,
ol {
  /* 箇条書きの「・」や番号を消す */
  list-style: none;
  margin: 0;
  padding: 0;
}

a {
  /* リンクの下線を消す */
  text-decoration: none;
  /* リンクの青色をやめ、親の文字色を継承 */
  color: inherit;
}

img {
  /* 画像が親より大きくならないように（はみ出し防止） */
  max-width: 100%;
  /* 縦横比を保ったまま縮小 */
  height: auto;
  /* 画像下の謎の隙間を消す（inline要素の癖を回避） */
  display: block;
}
```

- `ul, ol` … 箇条書きの記号（・や1.2.3.）を消し、ナビゲーションなどに使いやすくします。
- `a` … リンクの「下線」と「青文字」という既定スタイルを消し、必要な場所だけ後で付け直します (§ 11.7・§ 13)。

- `img` … `max-width: 100%; height: auto;` は **画像の鉄板**。親の幅に収まりつつ、縦横比が崩れません。`display: block` は、画像の下にできる謎の隙間（インライン要素の行ベースラインの癖）を消すおまじないです。本アプリのロゴ画像もこの恩恵を受けます。

11.5 CSS変数の定義（47～51行目）

▼ **このブロックがやること**: よく使う値（見出しサイズ）に名前を付けて、後でまとめて使い回せるようにします。これを **CSS変数（カスタムプロパティ）** と呼びます。

```
/* :root は html要素＝最上位。ここで定義すると全体で使える */
:root {
  /* 見出し1のサイズ（画面幅で伸縮、§6.2のclamp） */
  --heading1: clamp(1.5rem, 1.5rem + 3vw, 4rem);
  /* 見出し2のサイズ */
  --heading2: clamp(0.7rem, 0.8rem + 3vw, 1.5rem);
  /* 段落のサイズ */
  --paragraph: clamp(0.3rem, 0.6rem + 3vw, 1rem);
}
```

- `--名前: 値;` で変数を **定義** し、`var(--名前)` で **呼び出し** ます（次の § 11.6 で使用）。
- `:root`（ルート）は文書の最上位（html）を指す特別なセレクタ。ここで定義すると全画面で使えます。
- 値を1か所で管理できるので、「見出しサイズを変えたい」ときは、ここだけ直せば全体に反映されます。

用語: CSS変数（カスタムプロパティ） プログラミングの「変数（値に名前を付けて使い回す箱）」のCSS版。 `--` で始まる名前を自分で決めて定義します。

11.6 body・見出し・段落（53～81行目）

▼ **このブロックがやること**: 本アプリ全画面の「土台レイアウト」をここで決めています。背景色・中央寄せ・最低の高さ・横スクロール、そして見出し/段落のサイズ（§ 11.5の変数を使用）。

```

body {
  font-family: 'Arial', sans-serif;
  margin: 0;
  /* 画面の縁に10pxの余白 */
  padding: 10px;
  /* 薄い灰色の背景 */
  background-color: #f4f7f9;
  /* 中身をFlexboxで配置 */
  display: flex;
  /* 横方向：中央寄せ（コンテンツを画面中央に） */
  justify-content: center;
  /* 縦方向：上寄せ */
  align-items: flex-start;
  /* 最低でも画面の高さ分は確保（vh=画面高さの1%） */
  min-height: 100vh;
  /* 横にはみ出たらスクロールバーを出す */
  overflow-x: auto;
  /* テキストを折り返さない（表が崩れないように） */
  white-space: nowrap;
}

h1 {
  /* §11.5で定義した変数を使用 */
  font-size: var(--heading1);
  /* 下に1文字分の余白 */
  margin-bottom: 1rem;
  /* 中央寄せ */
  text-align: center;
}

h2 {
  font-size: var(--heading2);
  margin-bottom: 0.2rem;
  text-align: center;
}

p {
  font-size: var(--paragraph);
  text-align: center;
}

```

ここで本アプリの「全コンテンツが画面中央に配置される」基本レイアウトが決まっています。 `body` 自体をFlexboxにして `display: flex; justify-content: center;` で中央寄せ——これが本アプリ全画面の土台です。 `align-items: flex-start` で「縦は上寄せ」なので、コンテンツは上から並びます。

`min-height: 100vh` で「中身が少なくても画面の高さいっぱいは確保」、`overflow-x: auto` で「表などが横にはみ出たら横スクロールバーを出す」。`white-space: nowrap` は「文字を勝手に折り返さない」——表のセルが改行で崩れるのを防ぐ狙いです（§ 3⑩のログインボタンが折り返さないのもこの仲間の指定）。

11.7 リンクのホバー（83～89行目）

```
a {  
  /* リンクの下線を消す（§11.4で消したものの再指定） */  
  text-decoration: none;  
}  
  
a:hover {  
  /* ただしマウスを乗せた時だけ下線を出す */  
  text-decoration: underline;  
}
```

「普段は下線なし、ホバー時だけ下線」という、よくあるリンクの演出です。`text-decoration` は「文字の装飾（下線・取り消し線など）」。`none` = なし、`underline` = 下線。

11.8 form（91～109行目）

▼ **このブロックがやること:** すべてのフォームを「白背景・角丸・影付き・中央寄せのカード」に見せます。さらに、狭い画面では幅の制約を緩めます。§ 3③の `<form>` がこのスタイルになります。

```

form {
  /* 白背景 */
  background-color: #fff;
  padding: 0;
  /* 角を丸く（半径8px） */
  border-radius: 8px;
  /* 影：右下方向に薄い影（立体感） */
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
  /* 最小幅350px */
  min-width: 350px;
  /* 最大幅450px */
  max-width: 450px;
  /* 上下10px、左右auto＝横中央寄せ */
  margin: 10px auto;
  display: flex;
  /* 中身を縦並びに */
  flex-direction: column;
  box-sizing: border-box;
}

/* 画面幅768px以下のとき */
@media (max-width: 768px) {
  form {
    width: auto;
    /* 最小幅の制約を外し、狭い画面に対応 */
    min-width: unset;
  }
}

```

- `border-radius: 8px` … 角を半径8pxで丸める。カードらしい柔らかい印象に。
- `box-shadow: 0 4px 6px rgba(0,0,0,0.1)` … 「横0・下4px・ぼかし6px・黒の透明度10%」の影。カードを背景から浮き上がらせます。
- `min-width: 350px; max-width: 450px` … フォームの幅を350～450pxの範囲に。広すぎず狭すぎず。
- `margin: 10px auto` … 左右autoでフォームが画面中央に来ます（§5.2）。
- メディアクエリで、768px以下では `min-width` の制約を外し、小さな画面で横にはみ出さないようにしています。

11.9 table（111～129行目）

▼ このブロックがやること: 表（一覧画面で多用）の罫線を1本にまとめ、セルに余白と中央寄

せを付け、見出しセル（th）の白文字に黒い縁取りを付けて読みやすくします。

```
table {
  width: auto;
  /* セルの境界線を重ねて1本に（二重線を防ぐ） */
  border-collapse: collapse;
  margin-top: 10px;
  /* セル内で折り返さない */
  white-space: nowrap;
}

th,
td {
  /* 薄い灰色の枠線 */
  border: 1px solid #ddd;
  /* セル内の余白 */
  padding: 10px;
  /* 中央寄せ */
  text-align: center;
}

th {
  /* 見出しセルの文字は白 */
  color: white;
  font-weight: bold;
  text-shadow: -0.5px -0.5px 0 #555, 0.5px -0.5px 0 #555,
               /* 文字を4方向に縁取り（白文字を見やすく） */
               -0.5px 0.5px 0 #555, 0.5px 0.5px 0 #555;
}
```

- `border-collapse: collapse` … 隣り合うセルの罫線を **1本にまとめる** 必須テク。これが無いと罫線が二重線になり、もたついて見えます。
- `th, td` … 見出しセルとデータセルの両方に、薄い枠線・余白10px・中央寄せ。
- `th` の `text-shadow` … 同じ影を **4方向**（左上・右上・左下・右下に各0.5px）に重ねることで、**白文字に黒い縁取り** を作っています。背景色（後述の `th-collar` など）の上でも白文字が読めるようにする工夫です。一覧画面（`KosuList` 等）の表ヘッダはこれです（§13）。

用語: text-shadow（テキストシャドウ） 文字に影を付けるプロパティ。**横ずれ 縦ずれ ぼかし 色** の順。4方向に小さくずらした影を重ねると「縁取り（フチ文字）」になります。

11.10 ボタン群（131～275行目）

本アプリには色違いのボタンが **7種類** あります。構造はすべて同じで、**背景色とホバー色だけ** が違います。代表として **青ボタン** を全解説し、他は差分の表で示します。

▼ **このブロックがやること**: `class="blue_button"` を付けた要素を、青背景・白文字（縁取り付き）・角丸・指カーソル・ホバーで濃い青になるボタンにします。§ 3⑩のログインボタンがこれです。

```
.blue_button {
  /* 青背景 */
  background-color: #007bff;
  /* 白文字 */
  color: #fff;
  /* 文字の縁取り (§11.9と同じ) */
  text-shadow: -0.5px -0.5px 0 #555, 0.5px -0.5px 0 #555, -0.5px 0.5px 0 #555, 0.5px 0.5px 0 #555;
  /* 内側余白＝ボタンの厚み */
  padding: 10px;
  font-size: 1rem;
  /* 枠線なし */
  border: none;
  /* 角丸 */
  border-radius: 4px;
  /* マウスを乗せると指マークに（押せる合図） */
  cursor: pointer;
  /* 色変化を0.3秒かけて (§9.1) */
  transition: background-color 0.3s ease;
  /* 文字を横書きに固定 */
  writing-mode: horizontal-tb;
  /* 文字を折り返さない */
  white-space: nowrap;
  width: auto;
  max-width: 100%;
}

.blue_button:hover {
  /* ホバーで濃い青へ */
  background-color: #0056b3;
  text-decoration: none;
}
```

- `cursor: pointer` … マウスを乗せると指マーク（👉）に変わり、「押せる」と分かります。

- `writing-mode: horizontal-tb` … 文字を必ず横書きに（縦書きになる事故を防ぐ保険）。
- `white-space: nowrap` … ボタンの文字を途中で折り返さない。
- `:hover` で背景色だけ濃い青に変え、`transition` のおかげで0.3秒かけてじわっと変化します。

他の6つのボタンは **背景色とホバー色だけ違い、構造は同一** です（黄・橙・桃・灰には左右marginも追加）：

クラス	通常色	ホバー色	主な用途
<code>blue_button</code>	<code>#007bff</code> 青	<code>#0056b3</code>	標準アクション（ログイン等）
<code>yellow_button</code>	<code>#ffd700</code> 黄	<code>#ffa500</code>	注意・編集
<code>green_button</code>	<code>#0f0</code> 緑	<code>#32cd32</code>	登録・実行
<code>light_blue_button</code>	<code>#0ff</code> 水	<code>#0af</code>	検索・絞り込み
<code>orange_button</code>	<code>#f50</code> 橙	<code>#f10</code>	強い操作（左右marginあり）
<code>pink_button</code>	<code>#ff8bc5</code> 桃	<code>#ff1493</code>	補助（左右marginあり）
<code>gray_button</code>	<code>#656565</code> 灰	<code>#3a3a3a</code>	戻る・キャンセル（左右marginあり）

保守のヒント：「あるボタンの色を変えたい」なら、そのボタンクラスの `background-color` と `:hover` の `background-color` の2か所を直せばOK。全画面のそのボタンに反映されます。逆に言うと、ここを直すと **全画面に影響する** ので、特定画面だけ変えたいときは `.module.css` 側で上書きします（§13・§14）。

11.11 アラート表示（277～295行目）

▼ **このブロックがやること：** `role="alert"` を持つ `<div>`（エラーメッセージ）を、赤系の警告ボックスにし、先頭に  を自動で付けます。 `Login.tsx` のエラー表示（§3⑨）がこれです。

```

/* role属性が alert の div を狙う（属性セクタ） */
div[role="alert"] {
  /* 薄い赤背景 */
  background-color: #f8d7da;
  /* 濃い赤文字 */
  color: #721c24;
  /* 赤系の枠 */
  border: 1px solid #f5c6cb;
  border-radius: 4px;
  padding: 10px;
  /* 上0・左右auto(中央)・下16px */
  margin: 0 auto 16px auto;
  font-size: 0.9rem;
  /* 改行やスペースを保持して表示 */
  white-space: pre-wrap;
  /* 長い単語を折り返す */
  overflow-wrap: break-word;
  /* どこでも折り返してはみ出し防止 */
  word-break: break-all;
  /* 画面幅の90%まで */
  max-width: 90vw;
  box-sizing: border-box;
}

/* アラートの中身の「前」に */
div[role="alert"]::before {
  /* ⚠️ マークを自動で足す (§8.2の擬似要素) */
  content: "⚠️ ";
  font-size: 1.1rem;
}

```

- `div[role="alert"]` は **属性セクタ**。「`role` 属性の値が `alert` の div」だけを狙います。`Login.tsx` がエラー時に出す `<div role="alert">` (§3⑨) に、自動でこのスタイルが付きます。
- `white-space: pre-wrap` … サーバーから来たエラーメッセージの改行をそのまま表示。
- `word-break: break-all` … 長いURLなどでも、どこでも折り返してはみ出しを防ぐ。
- `::before` で中身の **前** に「⚠️」を自動で足す。HTMLには書かないのに警告マークが出るのは、この擬似要素のおかげです。

表示例（ログインで未登録の従業員番号を入れた場合のイメージ）：

⚠ 登録されていない従業員番号です。 | ← 薄い赤背景・赤文字

11.12 ナビゲーション（297～311行目）

```
nav {
  display: flex;
  /* メニューリンクを中央に */
  justify-content: center;
  /* リンク間の隙間15px */
  gap: 15px;
  margin-bottom: 20px;
}

nav a {
  background: none;
  padding: 0;
  border-radius: 0;
  /* メニューのリンクは下線付き */
  text-decoration: underline;
  font-weight: normal;
  transition: color 0.3s ease;
}
```

- `nav` … セマンティックな「メニュー領域」タグ。Flexboxで中央寄せ、リンク間に15pxの隙間。
- `nav a` … メニュー内のリンク。 `text-decoration: underline` で下線を付け（§ 11.4で全リンクの下線を消したので、ここで復活）、 `transition: color 0.3s` で色変化をなめらかに。各一覧画面はこの `nav` の中で、画面ごとの色（§ 13の `kosu-nav` など）を上塗りします。

11.13 日付入力の調整（313～342行目）

▼ このブロックがやること: `<input type="date">`（日付入力欄）のブラウザ既定のめちゃくちゃしたUI（カレンダーアイコン・上下ボタン・クリアボタン）を整理し、「入力欄のどこをクリックしてもカレンダーが開く」使い勝手にします。

```

input[type=date]::-webkit-calendar-picker-indicator {
  /* カレンダーアイコンを重ねて配置 */
  position: absolute;
  /* 入力欄全体に広げ */
  width: 100%;
  height: 100%;
  /* 透明にして（見えないがクリック範囲は残る） */
  opacity: 0;
}

input[type="date"]::-webkit-inner-spin-button {
  /* 上下ボタンを消す */
  -webkit-appearance: none;
}

input[type="date"]::-webkit-clear-button {
  /* クリアボタンを消す */
  -webkit-appearance: none;
}

input[type="date"] {
  /* 上の絶対配置(absolute)の基準点になる */
  position: relative;
}

.MuiPickersInputBase-root .css-15ni0jc {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  opacity: 0;
  /* 重なり順を前面に */
  z-index: 1;
  cursor: pointer;
}

```

- `-webkit-` は **Chrome / Edge / Safari 向けの接頭辞**（ベンダープレフィックス）。これらのブラウザ独自の部品を狙うために必要です。
- カレンダーアイコンを `opacity: 0`（透明）で入力欄全体に広げることで、「どこをクリックしてもカレンダーが開く」を実現。
- `position: relative`（基準）と `position: absolute`（その基準に対して重ねる）はセットで使います。

- 末尾の `.MuiPickersInputBase-root .css-15ni0jc` は、Material-UI（本アプリが使うUI部品ライブラリ。第6章）の日付ピッカーに同じ細工をしたものです。本アプリの日付検索（`KosuList` の検索バー等）の使い勝手のための調整です。

用語: z-index（ゼットインデックス） 要素の「重なり順（前後）」。数字が大きいほど手前に来ます。`z-index: 1` で、透明なクリック領域を前面に置き、確実にクリックを拾います。

これで `global.css` 全342行を読み切りました。本アプリの「全画面共通の見た目」——中央寄せレイアウト・カード状フォーム・色違いボタン7種・表・アラート・日付入力の細工——のすべてがここにあります。

12. 実コード全解説：Login.module.css 全100行を1行ずつ

次に、ログイン画面 **専用** のスタイル `frontend/src/styles/MainPage/Login.module.css` を全部読みます。`global.css`（共通）との **使い分け** が体感できます。

12.1 出現アニメーション（1～21行目）

▼ **このブロックがやること**：「下からふわっと現れる」アニメーション（`fadeInUp`）を定義し、ログイン画面全体を囲む箱（`login-wrapper`）に適用します。§ 3①の一番外側の `<div>` がこれです。

```

/* 「下からふわっと」アニメの定義 (§9.2) */
@keyframes fadeInUp {
  from {
    /* 開始：透明 */
    opacity: 0;
    /*      50px下にずらした位置から */
    transform: translateY(50px);
  }
  to {
    /* 終了：不透明 */
    opacity: 1;
    /*      定位置へ */
    transform: translateY(0);
  }
}

.login-wrapper {
  /* 画面全体を1秒かけて出現させる */
  animation: fadeInUp 1s ease-out;
  width: 100%;
  max-height: 100%;
  /* 画面幅を超えない */
  max-width: 100vw;
  /* 横はみ出しはスクロール */
  overflow-x: auto;
  overflow-y: auto;
  /* iOSでのスクロールを滑らかに */
  -webkit-overflow-scrolling: touch;
  /* 横中央寄せ */
  margin: 0 auto;
}

```

- `animation: fadeInUp 1s ease-out` … 上で定義した `fadeInUp` を、1秒・`ease-out`（最後ゆっくり）で再生。ページを開くと、この箱ごと **下から1秒かけてふわっと** 現れます。
- `max-width: 100vw` … 画面幅を絶対に超えない（横スクロール暴発の防止）。
- `-webkit-overflow-scrolling: touch` … iPhone/iPad (iOS) で、指でのスクロールを慣性付きで滑らかにする指定。

12.2 入力エリア search-bar (23～31行目)

▼ **このブロックがやること:** ラベル・入力欄・ボタンを「縦並び・中央寄せ・隙間15px」で配置します。§3⑤の `<div className={styles["search-bar"]}>` がこれです。

```

.search-bar {
  display: flex;
  /* 縦並び（ラベル→入力→ボタンを縦に） */
  flex-direction: column;
  justify-content: center;
  /* 中央寄せ */
  align-items: center;
  /* 各要素の間隔15px */
  gap: 15px;
  margin: 20px 0;
  flex-wrap: wrap;
}

```

⚠ **同名クラスの方向違いに注意:** ここ（ログイン）の `search-bar` は `flex-direction: column`（縦並び）ですが、一覧画面 `KosuList` の `search-bar` は `row`（横並び）です（§13）。CSS Modules（§14）のおかげで名前が衝突せず、画面ごとに別の意味を持てます。

12.3 ロゴ login-logo（33～40行目）

```

.login-logo {
  /* 最大400px だが画面が狭ければ90%まで（min関数） */
  max-width: min(400px, 90%);
  width: auto;
  /* 縦横比を保つ */
  height: auto;
  /* 横中央 */
  margin: 0 auto;
  /* 下に40pxの余白（入力エリアとの間隔） */
  margin-bottom: 40px;
  /* 画像を歪めず枠に収める */
  object-fit: contain;
}

```

- `min(400px, 90%)` … 「2つのうち小さい方」を採る（§6.2）。大画面では400px、小画面では画面幅の90%——どんな画面でもはみ出しません。
- `object-fit: contain` … 画像を **歪めずに** 枠に収める（縦横比キープ）。

12.4 入力欄とフォーカス（42～54行目）

▼ **このブロックがやること:** ログインの入力欄に余白・枠・角丸を付け、選択時（focus）に枠を青く光らせます。§ 11.3で `outline: none` した代わりに「選択中の合図」をここで作っています。

```
.search-bar input {
  /* 入力欄の内側余白（縦8px・横10px） */
  padding: 8px 10px;
  /* 灰色の枠 */
  border: 1px solid #999;
  border-radius: 4px;
  font-size: 1rem;
  /* 幅は「親の100%から60px引いた値」（calc関数） */
  width: calc(100% - 60px);
}

/* 入力欄が選択された時（§8.1の擬似クラス） */
.search-bar input:focus {
  /* 枠を青に */
  border-color: #08f;
  outline: none;
  /* 周囲にうっすら影＝光って見える */
  box-shadow: 0 0 3px #555;
}
```

- `calc(100% - 60px)` … **計算式**（§ 6.2）。「親の幅いっぱいから60px分だけ狭く」。左右に余白を残します。
- `:focus` … 入力欄をクリック（選択）すると、枠が青（`#08f`）になり、`box-shadow` でうっすら光ります。「今ここに入力できますよ」の視覚的な合図です。

12.5 ヘルプボタン（菱形）（56～100行目）

本アプリで最も凝ったスタイル。`Login.tsx` の `<Link className={styles["help-button"]} >`（§ 3⑪）を、**45度傾いた菱形に「？」が浮かぶボタン** にしています。

▼ **このブロックがやること:** 白い小さな四角を45度回して菱形にし、画面の右下に固定。**30秒後にふわっと出現** させる。中の「？」は文字だけ正位置に戻す。ホバーで青背景＋白文字に。

```

.help-button {
  display: flex;
  align-items: center;
  /* 中身 (?) を中央に */
  justify-content: center;
  width: 20px;
  height: 20px;
  /* 白背景 */
  background-color: #ffffff;
  /* 青い枠 */
  border: 2px solid #007bff;
  /* 45度回転＝四角が菱形に見える */
  transform: rotate(45deg);
  /* 画面に固定（スクロールしても動かない） */
  position: fixed;
  /* 画面下から30px */
  bottom: 30px;
  /* 画面右から30px */
  right: 30px;
  text-decoration: none;
  cursor: pointer;
  /* 最初は透明（見えない） */
  opacity: 0;
  /* 30秒後に出現（後述） */
  animation: fadeIn 0s linear 30.0s forwards;
}

@keyframes fadeIn {
  to {
    /* 透明→不透明 */
    opacity: 1;
  }
}

/* ホバー時は色のみ変更（動きはなし） */
.help-button:hover {
  /* ホバーで青背景に（菱形が塗りつぶされる） */
  background-color: #007bff;
}

/* 菱形の中身として「?」を作る */
.help-button::after {
  content: "?";
  /* 菱形は45度傾いているので、文字だけ-45度で正位置に戻す */
  transform: rotate(-45deg);
  color: #007bff;
  font-weight: bold;
  font-size: 20px;
}

```

```

}

/* ホバー時の中の文字色 */
.help-button:hover::after {
  /* ホバー時、中の「?」を白に */
  color: #ffffff;
}

```

ここには高度なテクニックが詰まっています：

- `transform: rotate(45deg)` で正方形を菱形に見せ、中の `::after` の `?` だけ `rotate(-45deg)` で **文字の向きを元に戻す**。これをしないと「?」が斜めになってしまいます。「箱を傾けたら、中の文字も傾くので、文字だけ逆に傾け直す」という発想です。
- `position: fixed; bottom: 30px; right: 30px;` で **画面の右下に貼り付け**。`fixed` はスクロールしても動かない固定配置（§ 11.13の `absolute` と違い、基準は「画面そのもの」）。
- `animation: fadeIn 0s linear 30.0s forwards` … 形は「`fadeIn` を 0秒（一瞬）で、`linear` で、**30秒の遅延後** に再生、`forwards`」。
- `0s` … アニメ自体の長さ（一瞬で透明→不透明）。
- `30.0s` … **遅延（delay）**。30秒待ってから始まる。
- `forwards` … アニメ終了後の状態（不透明）を **保ち続ける**。
- つまり「**ログイン画面を30秒見ていると、こっそり右下にヘルプが現れる**」仕掛けです。「すぐ出すと邪魔だが、迷っている人には助けを出したい」という配慮です。

global と module の使い分けがここで腑に落ちる：`help-button`（この画面だけの凝った装飾）は **module**、`blue_button`（全画面共通のボタン）は **global**。「**共通は global、専用は module**」という設計が、実コードで実感できます。

これで `Login.module.css` 全100行を読み切りました。

13. 実コード追加解説：KosuList / MainMenu / Pagination

`global.css`（共通）と `Login.module.css`（専用）の関係が分かったところで、他の代表的な実CSSを見て、パターンの理解を固めます。

13.1 KosuList.module.css (一覧画面) — 表と横並び検索バー

工数一覧 `frontend/src/styles/KosuPage/KosuList.module.css` から、特徴的な部分を読みます。

▼ **このブロックがやること:** 一覧画面全体を縦並び中央寄せにし、見出しやナビ・表ヘッダを「水色 (#0ff)」のテーマカラーで統一します。

```
.kosu-list-wrapper {
  /* Loginと同じ出現アニメ (各ファイルにコピーされている) */
  animation: fadeInUp 1s ease-out;
  display: flex;
  /* 縦並び */
  flex-direction: column;
  justify-content: top;
  /* 中央寄せ */
  align-items: center;
  height: 100vh;
  width: 100%;
  margin: 0 auto;
  -webkit-overflow-scrolling: touch;
}

.th-collar {
  /* 表ヘッダの背景を水色に (global.cssのthの白文字+縁取りが映える) */
  background-color: #0ff;
  text-shadow: -0.5px -0.5px 0 #555, 0.5px -0.5px 0 #555, -0.5px 0.5px 0 #555, 0.5px 0.5px 0 #555;
}

.status-ok {
  /* 「OK」状態を青太字で */
  color: blue;
  font-weight: bold;
}

.status-ng {
  /* 「NG」状態を赤太字で */
  color: red;
  font-weight: bold;
}
```

ポイント:

- `th-collar` は `global.css` の `th` (白文字+黒縁取り。§ 11.9) に **背景色だけを足す** module クラス。 `<th className={\ ${styles["th-collar"]} `}>` のように `global` と `module` を組み合わせて使います。「共通の土台+画面ごとの色」という典型パターンです。
- `status-ok` / `status-ng` は、工数の入力状況を「青=OK・赤=NG」で色分け。一覧の表で「未入力の人」をひと目で分かるようにする工夫です。

横並びの検索バーとレスポンス分岐：

```
.search-bar {
  display: flex;
  /* 横並び (Loginのsearch-barはcolumn=縦だった) */
  flex-direction: row;
  justify-content: center;
  align-items: center;
  gap: 15px;
  margin: 20px 0;
  /* 入りきらなければ折り返す */
  flex-wrap: wrap;
}

@media (max-width: 768px) {
  .search-bar {
    /* 狭い画面では縦並びに切り替え */
    flex-direction: column;
    gap: 10px;
  }
  .kosu-nav a {
    /* ナビ文字を小さく */
    font-size: 0.8rem;
  }
}
```

横並び→縦並びの切り替えが「レスポンス」の典型: PCでは検索条件を横一列に並べ (`row`)、スマホ幅 (768px以下) では縦積み (`column`) に切り替えます。同じ要素が画面幅で並び方を変える——これがメディアクエリ+Flexboxの王道です。

13.2 MainMenu.module.css (メニュー画面) — リンクをボタンに見せる

メニュー画面 `frontend/src/MainPage/MainMenu.tsx` は、 `<Link>` (本来はただのリンク) を **巨大なカラーボタン** に見せています。JSX側はこうです (§ 3で学んだ知識で読めます)：

```

{ /* リンクだが見た目はボタン */}
<Link to="/kosu-menu" className={styles["kosu-menu-button"]}>
  { /* 左にアイコン */}
  <img src={Note} alt="工数アイコン" className={styles["icon"]} />
  { /* 中央にラベル */}
  工数MENU
</Link>

```

対応するCSS（水色の「工数MENU」ボタン）：

▼ このブロックがやること: `<Link>` を、横幅最大400px・水色背景・アイコンとラベルを両端に配置した大きなボタンに見せます。

```

.kosu-menu-button {
  width: 100%;
  max-width: 400px;
  /* 水色 */
  background-color: #0ff;
  color: #fff;
  /* 画面幅で文字サイズ伸縮 (§6.2) */
  font-size: clamp(1rem, 1rem + 3vw, 2rem);
  border-radius: 4px;
  cursor: pointer;
  transition: background-color 0.3s ease, box-shadow 0.3s ease;
  display: flex;
  align-items: center;
  /* アイコンとラベルを両端に */
  justify-content: space-between;
  padding: 10px 15px;
  position: relative;
}
.kosu-menu-button:hover {
  /* ホバーで濃い水色 */
  background-color: #0af;
  /* 影で浮き上がる */
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}

```

`MainMenu` には同じ構造で色違いのボタンが6つ（工数=水色・定義区分=緑・人員=黄・班員=橙・問い合わせ=桃・管理者=灰）並びます。`global.css` のボタン群（§ 11.10）と発想は同じ「色違いの量産」です。

擬似要素を使った「左右対称」の工夫と、スマホでの非表示：

```
.kosu-menu-button::after,  
.def-menu-button::after,  
/* ...他のボタンも... */ {  
  /* 空の飾りを右端に置く */  
  content: "";  
  height: 1em;  
  flex: 0 0 auto;  
  /* 左のアイコンと同じ幅の「見えない箱」 */  
  width: var(--icon-width, 1em);  
}  
  
@media (max-width: 768px) {  
  .admin-menu-button {  
    /* 狭い画面では管理者ボタンを隠す */  
    display: none !important;  
  }  
}
```

- 左にアイコン、右に同じ幅の **空の `::after`** を置くことで、**`justify-content: space-between`** でもラベルが **見た目の中央** に来るようにしています（左右のバランス取り）。
- **`display: none !important`** … スマホ幅では管理者メニューを **完全に隠す**（管理者操作はPC前提のため）。**`!important`** で確実に隠す数少ない正当な使用例です（§ 4.3）。

13.3 Pagination.module.css（共通部品） — 矢印型ボタン

ページ送り部品 `frontend/src/styles/Components/Pagination.module.css` は、**`clip-path`**（クリップパス）で **矢印型（三角）のボタン** を作っています。

▼ **このブロックがやること**：「次へ」ボタンを、右向きの三角形（▶）に切り抜きます。
`clip-path` は要素を任意の多角形に切り抜くプロパティです。

```

.pagination {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 15px;
  margin-top: 20px;
  white-space: nowrap;
}

.pagination .next-button {
  position: relative;
  width: 70px;
  height: 50px;
  /* 色はJS側から変数で渡す */
  background-color: var(--btn-bg-color, #fff);
  border: none;
  /* 左上→右中央→左下＝右向き三角 */
  clip-path: polygon(0% 0%, 100% 50%, 0% 100%);
  cursor: pointer;
  transition: background-color 0.3s ease, box-shadow 0.3s ease;
  display: flex;
  align-items: center;
  justify-content: flex-start;
  padding-left: 10px;
  color: #FFF;
  font-weight: bold;
  text-shadow: -0.5px -0.5px 0 #555, 0.5px -0.5px 0 #555, -0.5px 0.5px 0 #555, 0.5px
0.5px 0 #555;
}

.pagination button:disabled {
  /* 押せないときは灰色に */
  background-color: #999;
  color: #666;
  /* カーソルが「禁止マーク」に */
  cursor: not-allowed;
}

```

- `clip-path: polygon(0% 0%, 100% 50%, 0% 100%)` … 3点（左上・右の中央・左下）を結んだ三角形に要素を切り抜く＝「▶」の形。「次へ」ボタンの矢印はこれで作っています。
- `var(--btn-bg-color, #fff)` … CSS変数を使い、色をJavaScript側から渡せるように。`, #fff`は「変数が未指定なら白」という既定値。
- `:disabled` (§ 8.1) … 最初のページで「前へ」が押せないとき、灰色＋禁止カーソルにして「押せない」と伝えます。`cursor: not-allowed` は禁止マーク（🚫）。

共通部品のCSSの考え方: `Pagination` は全画面で使い回す「部品」なので、色は固定せず **CSS変数で外から差し替えられる** 設計になっています。これにより、工数一覧では水色、人員一覧では黄色…と、画面ごとに同じ部品を別色で使えます。

14. CSS Modulesのしくみ

§ 3・§ 12・§ 13で見た `styles["login-wrapper"]` の正体を説明します。

用語: CSS Modules (シーエスエス・モジュールズ) とは? ファイルごとにCSSの効く範囲を **その画面だけに限定** するしくみ。ファイル名を `〇〇.module.css` にすると有効になります。

14.1 なぜ必要か

ふつうのCSS (`global.css` のような) は **全画面で共有** されます。すると、別々の画面で偶然同じクラス名 (例 `.search-bar`) を使うと **衝突** して、片方の見た目が壊れます。実際、本アプリには `Login` と `KosuList` の両方に `.search-bar` があり、しかも **片や縦並び・片や横並び** です (§ 12.2・§ 13.1)。これがもしふつうのCSSなら、後から読み込まれた方に上書きされて事故ります。

CSS Modules は、各クラス名をビルド (変換) 時に **一意 (重複しない) な名前へ自動改名** することで、これを防ぎます。

14.2 動作

```
// styles というオブジェクトに各クラスが入る
import styles from "../styles/MainPage/Login.module.css";
// ...
// styles["login-wrapper"] を適用
<div className={styles["login-wrapper"]}>
```

ビルド後、`login-wrapper` は例えば `Login_login-wrapper__a1B2c` のような **重複しない名前** に自動変換されます。`KosuList` の `search-bar` は `KosuList_search-bar__x9Y8z` に、`Login` の `search-bar` は `Login_search-bar__p3Q4r` に——別名になるので、絶対に衝突しません。

14.3 global との併用

1つの要素に、module クラスと global クラスを **両方** 付けられます。 `Login.tsx` がまさにそうです。

```
// global (文字列で直接)
<button type="submit" className="blue_button">ログイン</button>
// module (styles経由)
<div className={styles["search-bar"]}>
```

- **global** (`global.css`) … 文字列で直接 `className="blue_button"` 。 **全画面共通**。改名されない。
- **module** (`〇〇.module.css`) … `className={styles["..."]}` 。 **その画面専用**。自動改名される。

	global	module
ファイル名	<code>global.css</code>	<code>〇〇.module.css</code>
書き方	<code>className="名前"</code>	<code>className={styles["名前"]}</code>
効く範囲	全画面	そのファイルだけ
名前衝突	する (注意)	しない (自動改名)
用途	共通ボタン・表・アラート	各画面固有のレイアウト・色

⚠ よくあるミス: module のクラスを `className="login-wrapper"` (文字列) と書いてしまうと、改名後の本当の名前と一致せず **スタイルが効きません**。module は必ず `styles["login-wrapper"]` の形で書きます。逆に global を `styles["blue_button"]` と書いても、global.css は module ではないので効きません。

15. 開発者ツール (F12) 実践

見た目の調整は、ブラウザの **開発者ツール** で「現物を見ながら」進めるのが最速・最安全です。

用語: 開発者ツール（デベロッパーツール）とは？ ブラウザに最初から入っている「ページの中身を調べる・その場で実験する」ための道具一式。コードを保存せずに見た目を試せるので、壊す心配がありません。

15.1 開き方

- 画面で **右クリック** → 「**検証**」、または **F12** キー。

15.2 Elements（エレメンツ）タブ — HTML/CSSを調べる・試す

1. 左上の「矢印アイコン」を押し、調べたい部分（例：ログインボタン）をクリック。
2. 右側に、その要素に効いている **CSSルール** が一覧表示される。
3. 値をその場で書き換えると、画面が **即座に変化**（**保存はされない** ので、安心して実験できる。リロードで元に戻る）。
4. 効いているクラス名（例 `blue_button`）が分かったら、その名前で `global.css` や `.module.css` を **検索** して、本当の修正をする。

CSS Modules の改名を逆引きするコツ: module のクラスは `Login_login-wrapper__a1B2c` のような改名後の名前が表示されます。___ の前の `login-wrapper` の部分を見れば、元のファイル（Login）と元のクラス名が分かります。その「元の名前」で `.module.css` を検索すれば、修正箇所にとどりがけます。

15.3 Console（コンソール）タブ — エラーを見る

- JavaScriptのエラーが赤字で出ます。「画面が真っ白」「ボタンが効かない」等のとき、まずここを見ます。エラーの1行目をそのまま検索すると、原因が分かることが多いです。

15.4 Network（ネットワーク）タブ — 通信を見る

- APIの通信（リクエスト/レスポンス）が見られます。「保存したのにデータが更新されない」等の調査で、第7章（バックエンド）の知識と合わせて活用します。

保守の黄金フロー: 「① 直したい部分を右クリック→検証 → ② クラス名を特定 → ③ そのクラスをコードで検索（global か module か見極め） → ④ 修正 → ⑤ ブラウザで確認」。この順

番で進めれば、迷わず・壊さず直せます。

16. アクセシビリティ

用語: アクセシビリティ（accessibility：アクセシビリティ）とは？ 目の不自由な方や、マウスを使えない方など、**誰もが使えるようにする** 配慮のこと。「access（アクセス）+ ability（できること）」＝「誰でも使える度合い」。

本アプリにも配慮が入っています。

- **`<label htmlFor="numberInput">`** (§ 3⑥) … 入力欄に「これは従業員番号の欄」という意味を結びつけ、読み上げソフトが「従業員番号、入力欄」と説明できます。ラベルをクリックで入力欄にカーソルが入る利点も。
- **`<div role="alert">`** (§ 3⑨・§ 11.11) … **`role="alert"`** を付けると、読み上げソフトが**エラーを自動で読み上げ**ます。視覚に頼らずエラーに気づけます。
- **`alt="業務工数システムロゴ"`** (§ 3②) … 画像が表示できないときや読み上げ時に、代替の説明になります。**`alt=""`**（空）にすると「装飾画像なので読み飛ばす」の意味になり、これも適切な使い分けです。

⚠ 保守時の注意: 画面を直すときも、**ラベルと入力欄の結びつき（`htmlFor / id`）**と**画像の`alt`**を消さないようにしましょう。**`outline: none`** (§ 11.3) で選択枠を消した場合は、**`:focus`**の代替スタイル (§ 12.4) を必ず用意してください。さもないとキーボードだけで操作する人が「今どこにいるか」分からなくなります。

17. よくある落とし穴とトラブルシューティング

手を動かすとき、また保守でつまづいたとき用の早見表です。

症状	原因	対処
----	----	----

スタイルが効かない	module を文字列で書いた／ global を styles で書いた	<code>styles["名前"]</code> (module) と <code>"名前"</code> (global) の使い分けを確認 (§ 14.3)
module のクラスがそもそも効かない	クラス名のスペルミス、 <code>.module.css</code> でないファイル名	開発者ツールで改名後の名前が出ているか確認
幅がはみ出す・崩れる	<code>box-sizing</code> 未設定で padding が幅に加算	<code>box-sizing: border-box</code> (本アプリは全要素に設定済み・§ 11.1)
中央寄せできない	Flexの軸を勘違い (<code>row</code> / <code>column</code> で担当が逆)	<code>flex-direction</code> を確認し、 <code>justify-content</code> / <code>align-items</code> の担当方向を見直す (§ 7.2)
表に二重線が出る	罫線が重なっている	<code>border-collapse: collapse</code> (§ 11.9)
ホバーが効かない	<code>:hover</code> の対象セレクト違い	開発者ツールで実際のクラス名を確認 (§ 15)
余白が思った通りにならない	margin と padding の取り違い	margin=外側、padding=内側 (§ 5)
画像が縦長/横長に歪む	width と height を両方固定	<code>height: auto</code> で縦横比を保つ (§ 11.4)
<code>;</code> のつけ忘れて以降が壊れる	行末のセミコロン漏れ	各行の末尾 <code>;</code> を確認 (§ 4.1)
入力欄の選択枠が消えて不便	<code>outline: none</code> のみで代替なし	<code>:focus</code> で <code>border-color</code> / <code>box-shadow</code> を付ける (§ 12.4)
変更したのに反映されない	ブラウザのキャッシュ	スーパーリロード (Ctrl+Shift+R)

18. 演習問題

手を動かすと定着します。**ローカル環境** (自分のPC内・壊しても安全) で試してください。実験が終わったら必ず元に戻すこと。

1. **色を変える (global)** : `global.css` の `.blue_button` の通常色を緑 (`#0f0`) に変え、ログインボタンの色が変わることを確認。あわせて `MainMenu` のログアウトボタン (同じ `blue_button`) も変わることを確認 (global は全画面に効く実感)。終わったら元に戻す。

2. **余白を変える (module)** : `Login.module.css` の `.login-logo` の `margin-bottom` を `40px` → `80px` にして、ロゴと入力欄の間隔が広がることを確認。
3. **アニメ時間を変える** : `.login-wrapper` の `animation: fadeInUp 1s` を `3s` にして、出現がゆっくりになることを確認。
4. **ヘルプボタンを早く出す** : `Login.module.css` の `.help-button` の `animation` の遅延 `30.0s` を `2s` にして、2秒でヘルプが出ることを確認 (実験後は元に戻す)。
5. **開発者ツール** : F12でログインボタンを検証し、`background-color` をその場で赤に変えてみる (リロードで元に戻ることも確認)。
6. **軸の入れ替えを体感** : `Login.module.css` の `.search-bar` の `flex-direction: column` を `row` に変え、ラベル・入力欄・ボタンが横並びになることを確認 (§ 7.2の軸の話を実感)。
7. **レスポンスを観察** : F12で画面幅を狭く (768px以下に) して、`KosuList` の検索バーが横並びから縦並びに変わることを、`MainMenu` の管理者ボタンが消えることを確認 (§ 13)。
8. **属性セクタを確認** : 開発者ツールで `<div role="alert">` を探し、`div[role="alert"]` と `::before` のルールが効いて「⚠」が出ていることを確認 (§ 11.11)。

19. この章のまとめ

- 画面は **HTML (骨組み) + CSS (見た目) + JS (動き)** の3層。ブラウザはサーバーからHTMLを受け取り、**DOM (木構造)** を作り、CSSを当てて **レンダリング** する。JSがDOMを書き換えると再レンダリングされる。
- HTMLは **タグ** で構造を作り、**属性** で情報を足す。入れ子は交差させない。空要素はJSXでは `</>` で閉じる。本アプリは **フォーム** (form/label/input/button) が中心。JSXでは `class` → `className`、`for` → `htmlFor`、`{ }` でJSの値を埋め込む。
- CSSは **セクタ → プロパティ:値;。カスケード** (優先順位: `#id` > `.class` > タグ、同強さは後勝ち、`!important` は最強だが乱用禁物)。
- 要点プロパティ群: **ボックスモデル** (margin/border/padding/content) と `box-sizing: border-box`、**Flexbox** (`display: flex` + `flex-direction` + `justify-content` / `align-items`、軸の取り違えに注意)、単位 (px/rem/%/vw、`clamp` / `calc` / `min`)、色 (16進・rgba)、疑似クラス (`:hover` / `:focus` / `:disabled`)・疑似要素 (`::after` / `::before` + `content`)、トランジション・`@keyframes` アニメ、メディアクエリ。
- 本アプリの `global.css` **全342行** (全要素リセット・CSS変数・中央寄せレイアウト・カード状フォーム・表+縁取り白文字・色違いボタン7種・`role="alert"` + ⚠・日付入力の細工) と `Login.module.css` **全100行** (出現アニメ `fadeInUp`・入力フォーカス・30秒後に出る菱形ヘルプボタン) を1行ずつ読み切った。さらに `KosuList` (横並び検索バー+レスポンス+テーマ

色)・`MainMenu` (リンクを巨大カラーボタンに)・`Pagination` (`clip-path` で矢印ボタン) の
実例も確認した。

- **global = 全画面共通、module = その画面専用**。`CSS Modules` がクラス名を自動改名して名前衝突を防ぐ。1要素に両方を併用できる (例: `<button className="blue_button">` + `<div className={styles["search-bar"]}>`)。
- 見た目の調整・調査は **開発者ツール (F12) → クラス名特定 → コード検索 (global か module が見極め) → 修正 → 確認** の流れで。アクセシビリティ (`htmlFor` / `id` ・ `role="alert"` ・ `alt`) は消さない。

次は、画面に動きを与える言語、「04_JavaScript_TypeScript.md」へ進みます。ここで何度も出てきた `onChange` ・ `useState` ・ `Number()` ・ `{条件 && 表示}` といった「動きの部分」の正体が、いよいよ明らかになります。